Domain Rules

RoomieMatch – Requirements Document

Members: Max Siebengartner, Zain Rizvi

Domain Rules

1. A User must register with a name, email, password, age, gender, occupation, bio, and an optional profile photo before accessing the platform.
2. Every User has one UserPreferences record that stores their lifestyle habits: cleanliness level, sleep schedule, pet friendliness, smoking preference, noise tolerance, guest policy, and work schedule.
3. A User may post one or more Listings. Each Listing includes a title, description, rent price, location (address, city, state, zip code), available date, number of rooms, number of bathrooms, and an active/inactive status.
4. A Listing may display zero or more Photos. Each Photo stores an image URL and the time it was uploaded.
5. A User may make a UserInterest to express interest in a Listing. Each UserInterest stores the renter, the listing, a status (e.g., Pending, Accepted, Rejected), and the date the interest was created.
6. A Listing can receive interest from zero or more Users, and a User can make interest in zero or more Listings.r
7. A User may send and receive Messages in the context of a specific Listing. Each Message stores the sender, receiver, associated listing, message content, sent timestamp, and a read status.
8. A User may create a Report against another User or against a Listing. Each Report records the reporter, the reported user or listing, a written reason, and the time it was created.
9. A User can create zero or more Reports, and a User or Listing can be the subject of zero or more Reports.
10. All User data is retained even if an account is deactivated, preserving associated messages, interests, and reports.

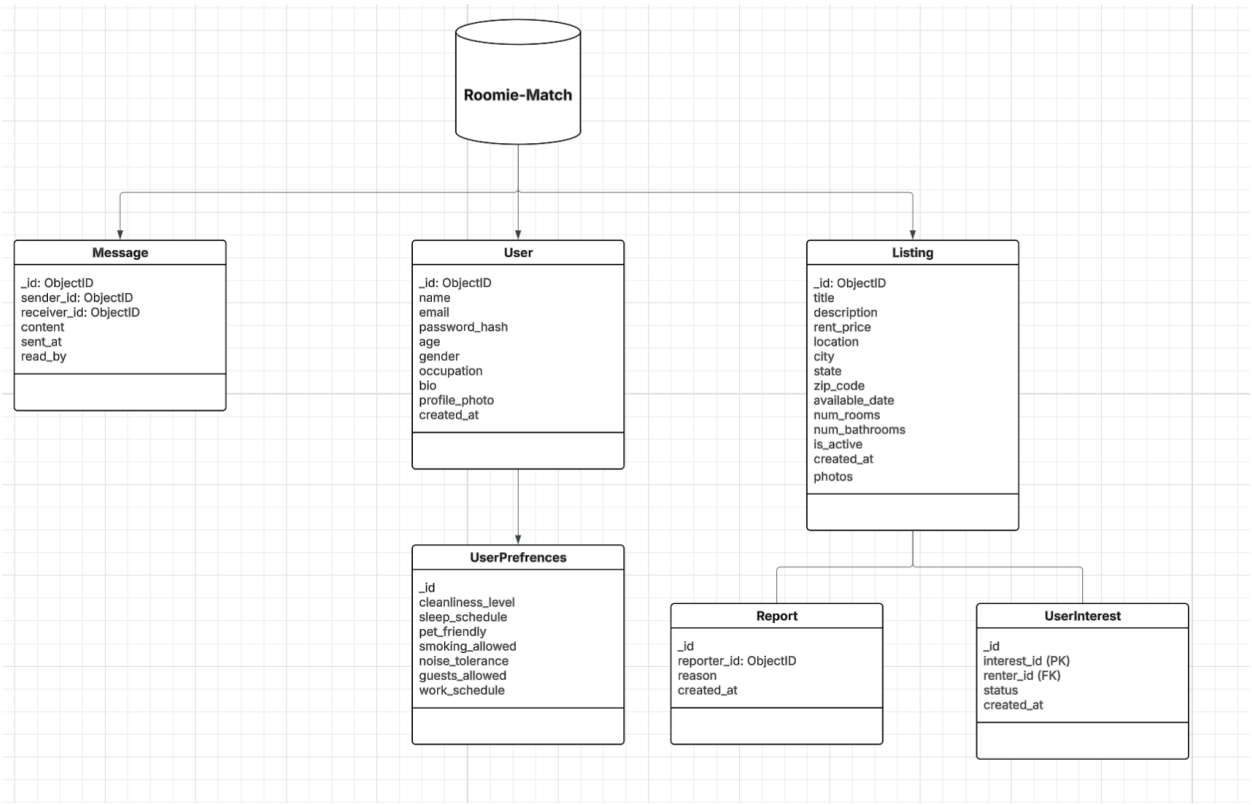
Nouns: User, UserPreferences, Listing, Photo, ListingPhoto, ProfilePhoto, UserInterest, Message, Report

Verbs:

- User **registers**
- User **has** UserPreferences
- User **posts** Listings
- Listing **displays** Photos
- User **makes** UserInterest (expresses interest in a Listing)
- User **sends/receives** Messages
- User **creates** Reports
- Data **is retained** on deactivation

Diagrams

UML Class



MongoDB Schemas

For this database, a three-collection system for Users, Listings, and Messages matches the app structure well, based on the growth rate of each data type, nested object structure, and independent querying. These are the following JSON schemas for each object type in the collections:

User JSON Schema

```
{
```

```
"$schema": "http://json-schema.org/draft-07/schema#",
"type": "object",
"properties": {
  "_id": {
    "type": "object",
    "description": "MongoDB ObjectId wrapper",
    "properties": {
      "$oid": {
        "type": "string",
        "description": "MongoDB ObjectId hex string (24 chars)",
        "pattern": "^[a-f0-9]{24}$"
      }
    },
    "required": ["$oid"]
  },
  "_supabase_id": {
    "type": "string",
    "description": "UUID v4 -- Supabase auth user ID",
    "format": "uuid"
  },
  "name": {
    "type": "string",
    "description": "Full display name"
  },
  "email": {
    "type": "string",
  },
  "password_hash": {
    "type": "string",
    "description": "bcrypt hash (prefix $2a$, $2b$, etc.)"
  },
  "age": {
    "type": "string",
    "pattern": "^[0-9]+$"
  },
  "gender": {
    "type": "string",
    "description": "Free-text gender identity"
  },
  "occupation": {
    "type": "string",
    "description": "Job title or role"
  },
```

```
    "bio": {
      "type": "string",
      "description": "Free-text user biography"
    },
    "profile_photo": {
      "type": "string",
      "description": "Base64-encoded image as a data URI",
      "pattern": "^data:image\\/[a-zA-Z]+; base64,"
    },
    "created_at": {
      "type": "string",
    },
    "preferences": {
      "type": "object",
      "description": "Roommate/lifestyle preferences",
      "properties": {
        "cleanliness_level": {
          "type": "string",
          "description": "Tidiness expectation",
          "enum": ["Very tidy", "Tidy", "Moderate", "Relaxed", "Messy"]
        },
        "sleep_schedule": {
          "type": "string",
          "description": "Typical sleep pattern",
          "enum": ["Early bird", "Night owl", "9-to-5", "Irregular"]
        },
        "pet_friendly": {
          "type": "string",
          "description": "Pet tolerance level",
          "enum": ["No pets", "Dogs ok", "Cats ok", "All pets ok"]
        },
        "smoking_allowed": {
          "type": "bool",
        },
        "noise_tolerance": {
          "type": "string",
          "description": "Acceptable noise level",
          "enum": ["Silent", "Quiet", "Moderate", "Lively"]
        },
        "guests_allowed": {
          "type": "string",
          "description": "Guest frequency preference",
          "enum": ["No guests", "Occasional guests", "Frequent guests"]
```

```
        },
        "work_schedule": {
          "type": "string",
          "description": "Work pattern",
          "enum": ["9-to-5", "Remote", "Night shift",
"Freelance/irregular"]
        }
      },
      "required": [
        "cleanliness_level", "sleep_schedule", "pet_friendly",
        "smoking_allowed", "noise_tolerance", "guests_allowed",
"work_schedule"
      ]
    }
  },
  "required": [
    "_id", "_supabase_id", "name", "email", "password_hash",
    "age", "gender", "occupation", "bio", "profile_photo",
    "created_at", "preferences"
  ]
}
```

Listing JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "_id": {
      "type": "object",
      "properties": {
        "$oid": {
          "type": "string",
          "pattern": "^[a-f0-9]{24}$"
        }
      },
      "required": ["$oid"]
    },
    "_supabase_id": {
```

```
      "type": "integer",
      "description": "Should likely be UUID v4 -- inconsistent with user
schema"
    },
    "user_id": {
      "type": "object",
      "description": "MongoDB ObjectId ref to owning user",
      "properties": {
        "$oid": {
          "type": "string",
          "pattern": "^[a-f0-9]{24}$"
        }
      },
      "required": ["$oid"]
    },
    "title": { "type": "string" },
    "description": { "type": "string" },
    "rent_price": {
      "type": "number",
      "minimum": 0
    },
    "location": { "type": "string" },
    "city": { "type": "string" },
    "state": {
      "type": "string",
      "pattern": "^[A-Z]{2}$"
    },
    "zip_code": {
      "type": "string",
      "description": "String to preserve leading zeros",
      "pattern": "^[0-9]{5}(-[0-9]{4})?$"
    },
    "available_date": {
      "type": "string",
      "description": "M/D/YYYY -- consider ISO 8601",
      "pattern": "^\\d{1,2}/\\d{1,2}/\\d{4}$"
    },
    "num_rooms": { "type": "integer", "minimum": 0 },
    "num_bathrooms": { "type": "integer", "minimum": 0 },
    "is_active": {
      "type": "string",
      "description": "'1'/'0' string -- consider boolean",
      "enum": ["0", "1"]
```

```
    },
    "photos": {
      "type": "array",
      "items": { "type": "string" }
    },
    "interests": {
      "type": "array",
      "description": "Shape unknown from empty array",
      "items": { "type": "object" }
    }
  },
  "required": [
    "_id", "_supabase_id", "user_id", "title", "description",
    "rent_price", "location", "city", "state", "zip_code",
    "available_date", "num_rooms", "num_bathrooms", "is_active",
    "photos", "interests"
  ]
}
```

Messages JSON Schema

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "properties": {
    "_id": {
      "type": "object",
      "properties": {
        "$oid": { "type": "string" }
      },
      "required": ["$oid"]
    },
    "sender_id": {
      "type": "object",
      "properties": {
        "$oid": { "type": "string" }
      },
      "required": ["$oid"]
    },
    "receiver_id": {
      "type": "object",
```

```
      "properties": {
        "$oid": { "type": "string" }
      },
      "required": ["$oid"]
    },
    "content": { "type": "string" },
    "sent_at": { "type": "string" },
    "read": { "type": "string", "enum": ["True", "False"] }
  },
  "required": ["_id", "sender_id", "receiver_id", "content", "sent_at",
"read"]
}
```

Redis Caching

Our most frequent calls on the last assignment were fetching the user authentication session and getting users by preferences. These would be relatively easy to cache and would only require saving the preference to a key such as “user:preferences:${userId}” with the preferences as a JSON string.

Another candidate query for caching is top listings by interest. This query uses the aggregation pipeline and it is expensive to redo $lookup, $match, and $sort on every call. Presumably with hundreds of users, this value wouldn't change too often, so we could cache it as a sorted set, where the values are the listing IDs and the scores are the number of interests.

Other queries like messaging seem cacheable, however the feature is built for live communication, so caching old messages would cause a noticeable delay, and the store would be invalidated often.

Queries

Find

User Authentication

```
export async function authenticateUser(email: string, password: string): Promise<{
success: boolean; data?: any; error?: string }> {
   const users = await usersCollection();
   if (!users) return { success: false, error: 'Failed to retrieve user' };

   try {
       const data = await users.findOne({ email, password_hash: password });
       if (!data) return { success: false, error: 'User not found' };
       return { success: true, data: rowToUser({ ...data, user_id:
userIdentifier(data) }) };
   } catch (error) {
       return { success: false, error: (error as Error).message };
   }
}
```

Loads the “users” Collection with the helper method. Then finds the first user document where the email and password hash match the filters. If nothing is returned, then no users match the entered credentials, and the login fails.

Fetching User Preferences

```
export async function getUserPreferences(userId: string): Promise<QueryResult> {
   const col = await usersCollection();
   if (!col) return { success: false, error: 'Failed to connect to database' };
   try {
       const oid = toObjectId(userId);
       if (!oid) return { success: false, error: 'Invalid user id' };
       const data = await col.findOne({ _id: oid }, { projection: { _id: 1,
preferences: 1 } });
       if (!data) return { success: false, error: 'User not found' };
       return { success: true, data:
[rowToUserPreferences(normalizePreferenceRow(data))] };
   } catch (e) {
       return { success: false, error: (e as Error).message };
   }
}
```

This function gets the nested UserPreference object inside a user document, using an _id based ‘find()’ on the collection.

Fetching a Text Exchange

```
export async function getConversationWithProfiles(userAId: string, userBId: string) {
   const messages = await messagesCollection();
   const users = await usersCollection();
   if (!messages || !users) throw new Error('Failed to connect to database');

   const aOid = new ObjectId(userAId);
   const bOid = new ObjectId(userBId);

   // Find 1: fetch both user profiles in a single query using $in
   const profiles = await users.find(
       { _id: { $in: [aOid, bOid] } },
       { projection: { _id: 1, name: 1, profile_photo: 1 } }
   ).toArray();

   // Index by string id for O(1) lookup when annotating messages
   const profileMap = new Map(
       profiles.map(p => [p._id.toString(), p])
   );

   // Find 2: all messages between the two users in either direction
   const thread = await messages.find(
       {
           $or: [
               { sender_id: aOid, receiver_id: bOid },
               { sender_id: bOid, receiver_id: aOid }
           ]
       },
       {
           projection: { content: 1, sent_at: 1, read: 1, sender_id: 1, receiver_id: 1
},
           sort: { sent_at: 1 }
       }
   ).toArray();

   // Annotate each message with sender/receiver profile data in JS
   // -- avoids a third round trip or a $lookup
   return thread.map(msg => ({
       ...msg,
       sender: profileMap.get(msg.sender_id.toString()) ?? null,
       receiver: profileMap.get(msg.receiver_id.toString()) ?? null
```

```
    }));
}
```

Gets all messages sent and received between two users, by performing a multi-collection find between users and messages. Each message is joined by its sender's name and profile photo, for easy identification and display.

Update

Editing an Existing Document

```
export async function updateListing(listingId: string, data: Partial<{
    title: string; description: string; rent_price: string; location: string;
    city: string; state: string; zip_code: string; available_date: string;
    num_rooms: string; num_bathrooms: string; is_active: string;
}>): Promise<QueryResult> {
    const col = await listingsCollection();
    if (!col) return { success: false, error: 'Failed to connect to database' };
    try {
        const oid = toObjectId(listingId);
        if (!oid) return { success: false, error: 'Invalid listing id' };
        const result = await col.updateOne({ _id: oid }, { $set: data });
        if (result.matchedCount === 0) return { success: false, error: 'Listing not
found' };
        return { success: true, data: { affected: result.modifiedCount } };
    } catch (e) {
        return { success: false, error: (e as Error).message };
    }
}
```

Updates a listing document by finding the listing document with a matching _id. Since the database is JSON on the application side, you can conveniently pass in a partial listing and update based on the non-null values.

Aggregation

Top Listings by Interest

```
db['listings'].aggregate([
  {
```

```
    $match: {
      $or: [
        { is_active: '1' },
        { is_active: 1 }
      ]
    }
  },
  {
    $addFields: {
      interestCount: { $size: { $ifNull: ['$interests', []] } }
    }
  },
  {
    $lookup: {
      from: 'users',
      localField: 'user_id',
      foreignField: '_id',
      as: 'owner'
    }
  },
  {
    $unwind: {
      path: '$owner',
      preserveNullAndEmptyArrays: true
    }
  },
  {
    $project: {
      title: 1,
      city: 1,
      state: 1,
      rent_price: 1,
      interestCount: 1,
      'owner.name': 1,
      'owner.profile_photo': 1
    }
  },
  { $sort: { interestCount: -1 } },
  { $limit: 10 }
])
```

Get listings by popularity, based on the number of interest objects they contain. It also filters active listings only, where { is_active: 1 }. The returned documents contain the listing information along with the owner's name and profile photo. This is useful for collecting housing metrics.

Unread Messages by Sender

```
db.collection('messages').aggregate([
  {
    // All messages involving this user (sent or received)
    $match: {
      $or: [
        { receiver_id: new ObjectId(currentUserId) },
        { sender_id: new ObjectId(currentUserId) }
      ]
    }
  },
  {
    // Normalize: identify the "other person" in each message
    $addFields: {
      partnerId: {
        $cond: {
          if: { $eq: ['$receiver_id', new ObjectId(currentUserId)] },
          then: '$sender_id',
          else: '$receiver_id'
        }
      },
      isUnread: {
        $and: [
          { $eq: ['$receiver_id', new ObjectId(currentUserId)] },
          { $eq: ['$read', 'False'] }
        ]
      }
    }
  },
  {
    $group: {
      _id: '$partnerId',
      unreadCount: { $sum: { $cond: ['$isUnread', 1, 0] } },
      lastMessageAt: { $max: '$sent_at' }
    }
  },
  {
```

```
    $lookup: {
      from: 'users',
      localField: '_id',
      foreignField: '_id',
      as: 'partner'
    }
  },
  { $unwind: '$partner' },
  {
    $project: {
      partnerId: '$_id',
      unreadCount: 1,
      lastMessageAt: 1,
      'partner.name': 1,
      'partner.profile_photo': 1
    }
  },
  { $sort: { lastMessageAt: -1 } }
])
```

Counts the number of unread messages per sender, and puts them in ascending order. The pipeline returns the messages with the sender's name and profile photo, so this could be used in a message preview or notification display.